

08 — Partial Indexes

"A partial index is a surgical index — it indexes only the rows you actually query."

Table of Contents

1. [Learning Objectives](#)
 2. [What is a Partial Index](#)
 3. [ASCII Diagram: Partial vs Full Index](#)
 4. [Storage Savings](#)
 5. [Query Planning with Partial Indexes](#)
 6. [Use Cases](#)
 7. [Partial Unique Indexes](#)
 8. [Combining Partial with Composite Indexes](#)
 9. [Creating Partial Indexes](#)
 10. [EXPLAIN Output](#)
 11. [Predicate Matching Rules](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions & Answers](#)
 16. [Exercises with Solutions](#)
 17. [Production Scenarios](#)
 18. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Explain what a partial index is and how it differs from a full index
 - Calculate the space savings from a partial index on a skewed dataset
 - Write partial index predicates that the query planner will match
 - Use partial unique indexes to enforce conditional uniqueness
 - Identify the use cases where partial indexes provide the greatest benefit
 - Understand when a query predicate must explicitly include the index predicate
-

What is a Partial Index

A **partial index** is an index built on only a **subset of rows** in a table — those satisfying a `WHERE` predicate specified at index creation time.

```
-- Full index: indexes ALL rows
CREATE INDEX idx_orders_customer ON orders(customer_id);

-- Partial index: only indexes rows where status = 'pending'
CREATE INDEX idx_orders_pending ON orders(customer_id)
WHERE status = 'pending';
```

The partial index is both **smaller** (fewer entries) and **more useful** for queries targeting the same subset of rows.

ASCII Diagram: Partial vs Full Index

Table: orders (10,000,000 rows)
status distribution:
 'pending': 10,000 rows (0.1%)
 'processing': 50,000 rows (0.5%)
 'completed': 9,940,000 rows (99.4%)

FULL INDEX on customer_id:

B-tree leaf pages: entries for ALL 10,000,000 rows
[cust=1, ctid] [cust=1, ctid] ... [cust=99999, ctid]
Size: ~160 MB
Contains 9,940,000 entries for 'completed' orders (rarely queried)

PARTIAL INDEX on customer_id WHERE status = 'pending':

B-tree leaf pages: entries for ONLY 10,000 'pending' rows
[cust=2, ctid] [cust=5, ctid] ... [cust=99998, ctid]
Size: ~160 KB
Contains ONLY the 10,000 operationally-relevant rows

Space savings: 160 MB → 160 KB = 1000× smaller!
Write overhead: Only 10,000 / 10,000,000 = 0.1% of INSERTs need to update the index
(only new 'pending' orders; completed orders don't touch the partial index)

Storage Savings

The storage savings depend on the selectivity of the partial index predicate.

Formula

Partial index size = Full index size × (rows matching predicate / total rows)

Example Calculations

Table Size	Predicate Selectivity	Full Index	Partial Index	Savings
100M rows	5% (active users)	1.6 GB	80 MB	95%
100M rows	1% (pending orders)	1.6 GB	16 MB	99%
100M rows	0.1% (flagged records)	1.6 GB	1.6 MB	99.9%
50M rows	10% (last 30 days)	800 MB	80 MB	90%

The more selective the predicate, the greater the savings.

Real-World Measurement

```
-- Compare sizes of two hypothetical indexes:
SELECT
    indexname,
    pg_size_pretty(pg_relation_size(indexrelid)) AS size,
    idx_scan
FROM pg_stat_user_indexes
WHERE relname = 'orders'
ORDER BY pg_relation_size(indexrelid) DESC;
```

Query Planning with Partial Indexes

The query planner uses a partial index only when the query's WHERE clause is **provably compatible** with the index predicate.

The Rule

For a partial index with predicate `WHERE P`, a query with predicate `WHERE Q` uses the index when `Q` logically **implies** `P` — i.e., every row satisfying `Q` also satisfies `P`.

Index predicate `P`: `status = 'pending'`

Query WHERE clause `Q`: `status = 'pending'`

`Q` implies `P`: YES → planner can use the index

Query WHERE clause `Q`: `status = 'pending' AND customer_id = 42`

`Q` implies `P`: YES (more restrictive than `P`) → planner can use the index

Query WHERE clause `Q`: `status IN ('pending', 'processing')`

`Q` does NOT imply `P` (rows with `status='processing'` satisfy `Q` but not `P`) → cannot use index

Query WHERE clause `Q`: (no WHERE clause)

`Q` does NOT imply `P` → cannot use index

Examples

```
-- Index:
CREATE INDEX idx_orders_pending_cust ON orders(customer_id)
WHERE status = 'pending';

-- Uses the index:
SELECT * FROM orders WHERE status = 'pending' AND customer_id = 42;
SELECT * FROM orders WHERE customer_id = 42 AND status = 'pending';
SELECT * FROM orders WHERE status = 'pending' ORDER BY customer_id;

-- Does NOT use the index:
```

```
SELECT * FROM orders WHERE customer_id = 42; -- status not constrained
SELECT * FROM orders WHERE status IN ('pending','processing') AND customer_id = 42;
SELECT * FROM orders WHERE status != 'completed' AND customer_id = 42;
-- (these queries may still benefit from a different index)
```

Use Cases

1. Hot Subset Queries (queue processing)

```
-- Table: jobs (99% completed, 1% pending)
CREATE INDEX CONCURRENTLY idx_jobs_pending
ON jobs(created_at, priority)
WHERE status = 'pending';

-- Queue worker query:
SELECT * FROM jobs
WHERE status = 'pending'
ORDER BY priority DESC, created_at ASC
LIMIT 10 FOR UPDATE SKIP LOCKED;
-- Uses tiny partial index instead of huge full index
```

2. Soft Delete Pattern

```
-- Table: users with soft delete (deleted_at IS NULL = active)
CREATE INDEX CONCURRENTLY idx_active_users_email
ON users(email)
WHERE deleted_at IS NULL;

-- Login query:
SELECT * FROM users WHERE email = 'alice@example.com' AND deleted_at IS NULL;
-- Only active users are in the index (typically 90%+ in most applications)
```

3. NULL-excluding Indexes

```
-- Column phone_number is often NULL; queries always filter for non-NULL
CREATE INDEX CONCURRENTLY idx_users_phone
ON users(phone_number)
WHERE phone_number IS NOT NULL;

-- Index excludes NULL rows, reducing size
SELECT * FROM users WHERE phone_number = '+1-555-0100';
-- Planner infers phone_number IS NOT NULL from the equality predicate
```

4. Recent Data Pattern

```

-- Frequently query data from the last 90 days
CREATE INDEX CONCURRENTLY idx_events_recent_user
ON events(user_id, created_at)
WHERE created_at > NOW() - INTERVAL '90 days';
-- WARNING: This predicate is not stable – the index doesn't auto-update
-- as time passes. Use a timestamp literal, not NOW():
CREATE INDEX CONCURRENTLY idx_events_recent_user
ON events(user_id, created_at)
WHERE created_at > '2024-01-01'; -- must manually rebuild when cutoff moves
-- Better: Use table partitioning instead for time-based data

```

5. Flag Columns

```

-- Table: newsletter_subscribers; flag = needs_welcome_email (rare)
CREATE INDEX CONCURRENTLY idx_subscribers_welcome
ON newsletter_subscribers(created_at)
WHERE needs_welcome_email = TRUE;

-- Cron job query:
SELECT * FROM newsletter_subscribers
WHERE needs_welcome_email = TRUE
ORDER BY created_at
LIMIT 100;
-- Tiny partial index, nearly instant

```

Partial Unique Indexes

Partial unique indexes enforce uniqueness only on rows matching the predicate.

Use Case: Unique active record per user

```

-- Allow multiple soft-deleted subscription records but only one active per user
CREATE UNIQUE INDEX idx_subscriptions_active_unique
ON subscriptions(user_id)
WHERE status = 'active';

-- This is allowed (different users):
INSERT INTO subscriptions(user_id, status) VALUES (1, 'active'), (2, 'active');

-- This is allowed (same user, but one is inactive):
INSERT INTO subscriptions(user_id, status) VALUES (1, 'inactive');

-- This is NOT allowed (second active subscription for same user):
INSERT INTO subscriptions(user_id, status) VALUES (1, 'active');
-- ERROR: duplicate key value violates unique constraint

```

Use Case: One unpaid invoice per customer

```
CREATE UNIQUE INDEX idx_invoices_one_unpaid
ON invoices(customer_id)
WHERE payment_status = 'unpaid';
```

Use Case: Unique non-null values

```
-- Allow multiple NULL values but enforce uniqueness among non-NULL
CREATE UNIQUE INDEX idx_users_invite_code
ON users(invite_code)
WHERE invite_code IS NOT NULL;
-- Standard UNIQUE index allows multiple NULLs too, but this makes intent explicit
```

Combining Partial with Composite Indexes

Partial and composite index features can be combined for maximum specificity.

```
-- Partial + composite + covering
CREATE INDEX CONCURRENTLY idx_active_orders_lookup
ON orders(customer_id, created_at DESC)
INCLUDE (total_amount, item_count)
WHERE status IN ('pending', 'processing');

-- Query that fully leverages this index:
SELECT customer_id, created_at, total_amount, item_count
FROM orders
WHERE status IN ('pending', 'processing')
      AND customer_id = 42
ORDER BY created_at DESC
LIMIT 10;
-- Index Only Scan using the partial composite covering index
```

Creating Partial Indexes

```
-- Basic partial index
CREATE INDEX idx_name ON table_name(column)
WHERE predicate;

-- Concurrent creation
CREATE INDEX CONCURRENTLY idx_orders_pending ON orders(customer_id)
WHERE status = 'pending';

-- Partial unique index
CREATE UNIQUE INDEX idx_users_active_email ON users(email)
WHERE is_deleted = FALSE;

-- Partial composite index
```

```
CREATE INDEX CONCURRENTLY idx_large_orders ON orders(customer_id, total DESC)
WHERE total > 1000;

-- Partial expression index
CREATE INDEX CONCURRENTLY idx_active_users_lower_email ON users(lower(email))
WHERE is_active = TRUE;

-- Partial index with complex predicate
CREATE INDEX CONCURRENTLY idx_priority_jobs ON jobs(queue_name, priority DESC)
WHERE status = 'queued' AND retry_count < 3;
```

EXPLAIN Output

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders
WHERE status = 'pending' AND customer_id = 42;
```

```
Index Scan using idx_orders_pending_cust on orders
  (cost=0.29..8.31 rows=3 width=128)
  (actual time=0.023..0.034 rows=3 loops=1)
Index Cond: (customer_id = 42)
Filter: (status = 'pending')      ← partial index predicate shown as Filter
Rows Removed by Filter: 0
Buffers: shared hit=5
Planning Time: 0.156 ms
Execution Time: 0.056 ms
```

Note: The partial index predicate `status = 'pending'` appears as a `Filter` condition because it was used to select which index to use, not as a further scan condition.

When the partial index is NOT used:

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
-- No partial index used (query doesn't imply status = 'pending')
```

```
Seq Scan on orders  (cost=0.00..234567.00 rows=50 width=128)
  Filter: (customer_id = 42)
-- or: Index Scan using some other index on customer_id
```

Predicate Matching Rules

The query planner performs **predicate implication checking** to determine if a query can use a partial index.

Simple Equality

```
-- Index: WHERE status = 'active'
WHERE status = 'active'      -- ✓ exact match
```

```

WHERE status = 'active' AND id > 100      -- ✓ more restrictive
WHERE status = 'active' OR id > 100      -- ✗ OR makes it less restrictive
WHERE status != 'inactive'               -- ✗ planner can't infer status='active'

```

IS NULL / IS NOT NULL

```

-- Index: WHERE email IS NOT NULL
WHERE email IS NOT NULL                  -- ✓ direct match
WHERE email = 'foo@bar.com'              -- ✓ equality implies IS NOT NULL
WHERE email LIKE 'foo%'                  -- ✓ LIKE implies IS NOT NULL
WHERE email IS NULL                      -- ✗ opposite condition

```

Comparisons

```

-- Index: WHERE amount > 1000
WHERE amount > 1000                      -- ✓
WHERE amount > 1500                      -- ✓ (more restrictive)
WHERE amount > 500                       -- ✗ (less restrictive – 500-1000 not in
index)
WHERE amount BETWEEN 1500 AND 2000       -- ✓ (implies amount > 1000)

```

Parameterized Queries

```

-- Be careful with bind parameters!
-- Index: WHERE status = 'pending'
SELECT * FROM orders WHERE status = $1; -- planner may or may NOT use the index
-- If $1 is 'pending' at runtime, the index should be used
-- The planner assumes generic parameter values, so it may plan generically
-- To force use: SET enable_seqscan = off (for testing only)

```

Common Mistakes

1. Using a volatile function in the predicate

```

-- WRONG: NOW() changes, index predicate is not stable
CREATE INDEX ON events(user_id)
WHERE created_at > NOW() - INTERVAL '30 days';
-- The predicate is re-evaluated at index creation time and never changes
-- After 30 days, this index becomes less and less useful

```

2. Expecting the planner to use the index without the predicate

```

-- Index: WHERE status = 'pending'
-- Query does NOT use the index:

```



```
SELECT * FROM orders WHERE customer_id = 42;
-- You must include the predicate in the query
```

3. Creating partial indexes that don't reflect actual query patterns

```
-- If queries are: WHERE status = 'pending' AND priority = 'high'
-- This partial index is not optimal:
CREATE INDEX ON orders(customer_id) WHERE status = 'pending';
-- Better:
CREATE INDEX ON orders(customer_id) WHERE status = 'pending' AND priority = 'high';
```

4. Not maintaining timestamp-based partial indexes

```
-- An index with WHERE created_at > '2024-01-01' becomes less useful over time
-- as more data falls within the predicate window
-- Plan: rebuild the index periodically or use table partitioning instead
```

Best Practices

1. **Use partial indexes for "hot subsets"** — active records, unprocessed queue items, flagged rows that represent a small fraction of the total.
2. **Combine partial with composite and covering** for maximum efficiency on critical queries.
3. **Use partial unique indexes** to enforce business rules like "only one active subscription per user" rather than complex application logic.
4. **Avoid dynamic predicates** (like `NOW()`) in index definitions — they create stale indexes.
5. **Document the business reason** for the predicate — future maintainers need to understand why certain rows are excluded.
6. **Measure savings:** Run `SELECT pg_size_pretty(pg_relation_size('your_index'))` before and after converting a full index to a partial index.
7. **Check predicate selectivity:** If the predicate matches 80%+ of rows, the savings are minimal. Target predicates matching < 20% of rows for meaningful benefit.

Performance Considerations

Write Overhead Reduction

A partial index only needs to be updated when an INSERT/UPDATE/DELETE affects rows matching the predicate.

```
Table: jobs (10M rows, 1% pending)
Full index on status+job_type: updated on EVERY insert (~100% of writes)
Partial index WHERE status='pending': updated ONLY when status='pending' inserts (~1%
```

```
of writes)
→ 99% reduction in index write overhead!
```

Cache Efficiency

A small partial index fits entirely in `shared_buffers`, reducing disk I/O:

```
Full index (160 MB): doesn't fit in typical 128 MB shared_buffers
Partial index (160 KB): fits in a single buffer pool page
```

Parallel Index Creation

Partial indexes can be created with `CREATE INDEX CONCURRENTLY`, and since they're smaller, concurrent creation is faster.

Interview Questions & Answers

Q1. What is a partial index and when should you use one?

A: A partial index is an index that covers only a subset of rows in a table, defined by a WHERE predicate in the `CREATE INDEX` statement. It should be used when: (1) Queries frequently filter on a specific subset of rows (e.g., `status='pending'`, `deleted_at IS NULL`). (2) That subset represents a small fraction of total rows (high predicate selectivity). (3) The full-table index would be unnecessarily large because most indexed rows are rarely queried. The benefits are: smaller index (storage savings), faster query performance (smaller tree = fewer I/Os), and reduced write overhead (index only updated for matching rows).

Q2. How does the query planner determine whether a query can use a partial index?

A: The planner performs predicate implication checking. For a partial index with predicate P and a query with WHERE clause Q, the index can be used if Q logically implies P — every row satisfying Q must also satisfy P. For example, a partial index `WHERE status='pending'` can be used by a query `WHERE status='pending' AND customer_id=42` because the query is more restrictive (implies the predicate). But a query `WHERE customer_id=42` (without the status constraint) cannot use the partial index because it doesn't imply `status='pending'`.

Q3. What is a partial unique index and when is it useful?

A: A partial unique index enforces uniqueness only among rows that satisfy the index predicate. This implements conditional uniqueness constraints that standard `UNIQUE` can't express. Common use cases: (1) Soft delete — allow multiple deleted records but only one active: `UNIQUE ON (user_id) WHERE status='active'`. (2) Nullable unique columns — enforce uniqueness for non-null values only: `UNIQUE ON (invite_code) WHERE invite_code IS NOT NULL`. (3) Business rules like "one unpaid invoice per customer": `UNIQUE ON (customer_id) WHERE paid=false`. Standard `UNIQUE` constraints would prohibit all duplicates including legitimate historical records.

Q4. Why is using NOW() in a partial index predicate problematic?

A: A partial index predicate is evaluated once at index creation time and remains fixed. `CREATE INDEX ON events(user_id) WHERE created_at > NOW() - INTERVAL '30 days'` creates an index for rows that existed in the last 30 days at creation time. As time passes, new rows inserted after index creation that fall within the rolling 30-day window are NOT included in the index (the predicate boundary is static). After 30

days, the index is completely stale. The fix is: use a fixed timestamp literal that you manually advance (requiring index rebuilds), or better — use table partitioning for time-based queries.

Q5. Can a partial index be combined with expression indexes and INCLUDE columns?

A: Yes. All index features can be combined:

```
CREATE INDEX ON users(lower(email)) -- expression index
INCLUDE (id, created_at)           -- covering columns
WHERE is_active = TRUE              -- partial predicate
AND deleted_at IS NULL;            -- compound predicate
```

This creates a partial expression covering index. It will be used by queries like: `SELECT id, created_at FROM users WHERE lower(email) = 'foo@bar.com' AND is_active = TRUE` and will return results without a heap fetch if id and created_at are all that's needed.

Exercises with Solutions

Exercise 1

A `tasks` table has 50M rows. 99% are `status = 'done'`. Workers query: `SELECT * FROM tasks WHERE status = 'todo' ORDER BY priority DESC LIMIT 20 FOR UPDATE SKIP LOCKED`. Design the optimal index.

Solution:

```
-- Partial composite index on the 'todo' subset only
CREATE INDEX CONCURRENTLY idx_tasks_todo
ON tasks(priority DESC)
WHERE status = 'todo';

-- This index is tiny (1% of 50M = 500K rows), fits in cache
-- Supports ORDER BY priority DESC (already sorted in index)
-- Enables fast SKIP LOCKED queue processing

-- Verify:
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM tasks
WHERE status = 'todo'
ORDER BY priority DESC
LIMIT 20
FOR UPDATE SKIP LOCKED;
-- Expected: Index Scan using idx_tasks_todo, very fast
```

Exercise 2

Design a partial unique index to enforce "each user can have at most one 'active' subscription, but can have multiple 'cancelled' subscriptions."

Solution:

```

CREATE TABLE subscriptions (
  id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  plan TEXT NOT NULL,
  status TEXT NOT NULL DEFAULT 'active',
  created_at TIMESTAMPTZ DEFAULT now(),
  cancelled_at TIMESTAMPTZ
);

-- Partial unique: unique active subscription per user
CREATE UNIQUE INDEX idx_subscriptions_one_active
ON subscriptions(user_id)
WHERE status = 'active';

-- Test:
INSERT INTO subscriptions(user_id, plan, status) VALUES (1, 'pro', 'active'); -- OK
INSERT INTO subscriptions(user_id, plan, status) VALUES (1, 'basic', 'cancelled');
-- OK
INSERT INTO subscriptions(user_id, plan, status) VALUES (1, 'enterprise', 'active');
-- ERROR: duplicate key value violates unique constraint

```

Production Scenarios

Scenario 1: E-mail Queue System

A transactional email system has an `email_queue` table with 500M rows (mostly sent). Only ~10,000 rows are `status='queued'` at any time. The queue worker was doing sequential scans and processing only 50 emails/second.

```

-- Problem: Full index on (status, scheduled_at) was 8 GB
-- Most of the 8 GB was for 'sent' emails never queried again

-- Solution: Partial index
CREATE INDEX CONCURRENTLY idx_email_queue_pending
ON email_queue(scheduled_at, priority DESC)
WHERE status = 'queued';

-- Size: 10K rows × ~20 bytes = ~200 KB (vs 8 GB before)
-- Queue worker performance: 50/s → 50,000/s (index fits in cache)

-- Worker query:
SELECT * FROM email_queue
WHERE status = 'queued'
  AND scheduled_at <= NOW()
ORDER BY priority DESC, scheduled_at
LIMIT 100
FOR UPDATE SKIP LOCKED;

```

Cross-References

- [01_index_fundamentals.md](#) — Index fundamentals
- [09_composite_indexes.md](#) — Composite index design
- [10_covering_indexes.md](#) — INCLUDE columns
- [11_expression_indexes.md](#) — Expression indexes
- [../08_Query_Optimization/03_scan_types.md](#) — Scan types